Chapter 3

Synthesizing Probabilistic Programs in Domain-Specific Modeling Languages

> In the earliest days, symbolic manipulation and abstract languages for knowledge representation and reasoning were considered the heart of intelligence. And I think in many ways they are still the best idea that anybody has ever had about how intelligence works in computational terms.
>
> Joshua B. Tenenbaum

Chapter 2 showed an informal tutorial of using Bayesian inference to synthesize probabilistic programs in specialized data modeling languages for time series, which help automate the process of learning interpretable and accurate models from data. This chapter formally describes the general "Bayesian synthesis" framework illustrated in Figure 1.2 from Chapter 1. Section 3.1 introduces probabilistic domain-specific languages for representing families of generative models. Each expression in the DSL is assigned two denotational semantics: one for the prior probability distribution of the expression and another for probability distribution over datasets that it induces. Section 3.2 formalizes the problem of Bayesian synthesis and identifies sufficient conditions needed for this problem to well defined. Section 3.3 describes a class of Markov chain Monte Carlo (MCMC) and sequential Monte Carlo (SMC) algorithm templates that deliver sound approximate solutions to the Bayesian synthesis problem. Section 3.4 introduces a family of probabilistic DSLs that are generated by context-free grammars, outlines a generic algorithm that implements both the MCMC and SMC algorithm templates for this DSL family, and proves that it satisfies the preconditions for sound inference, i.e., the solutions converge asymptotically to the posterior distribution over DSL expressions given the data. Section 3.5 revisits the univariate time series DSL from Chapter 2, verifying that it meets the conditions needed for sound synthesis. Section 3.6 discusses related work.

3.1 Probabilistic Domain-Specific Modeling Languages

Definition 3.1. A *domain-specific data modeling language* is comprised of:

- A countable set S called the *structure space*.
- An indexed family $\{\Theta_s, s \in A\}$ of sets, called the *parameter spaces*.
- A set T called the *input space*.
- An indexed family $\{X_t, t \in T\}$ of sets, called the *data spaces*. «

Definition 3.2. The set of *expressions* of a domain-specific data modeling language D is

$$\mathcal{L}(D) ::= \{(s, \theta) \mid s \in S, \theta \in \Theta_s\}. \tag{3.1}$$

For any expression $E ::= (s, \theta) \in \mathcal{L}(D)$, the symbol s denotes the *structure* of E and θ denotes the *parameter* of E. When D is clear from context, $\mathcal{L}(D)$ is written as $\mathcal{L}$. «

Definition 3.3. The set of *observations* of a domain-specific data modeling language D is

$$\mathcal{X}(D) ::= \{(t, x) \mid t \in T, x \in X_t\}. \tag{3.2}$$

For any observation $O ::= (t, x) \in \mathcal{X}(D)$, the symbol t denotes the *input* of O and x denotes the *output* of O. When D is clear from context, $\mathcal{X}(D)$ is written as $\mathcal{X}$. «

Definition 3.4. A *probabilistic domain-specific data modeling language*, or simply a probabilistic DSL, is a domain-specific data modeling language equipped with:

- For each $s \in S$, a sigma-algebra $\mathcal{F}_s$ over Θ_s and a sigma-finite base measure λ_s over $(\Theta_s, \mathcal{F}_s)$.
- For each $t \in T$, a sigma-algebra $\mathcal{F}_t$ over X_t and a sigma-finite base measure λ_t over $(X_t, \mathcal{F}_t)$.
- A semantic function $\text{Prior} : \mathcal{L} \to \mathbb{R}_{\geq 0}$.
- A semantic function $\text{Likelihood} : \mathcal{L} \to \mathcal{X} \to \mathbb{R}_{\geq 0}$. «

Definition 3.5. Following Fremlin [2009, 214L], for a probabilistic DSL D, the "disjoint-sum" sigma-algebra $\Sigma_{\mathcal{L}}$ over $\mathcal{L}$ has events of the form $\{(s, \theta) \mid s \in A, \theta \in A_s\}$ for some $A \subset S$ and $A_s \in \mathcal{F}_s$ for each $s \in A$. A generic event in $\Sigma_{\mathcal{L}}$ is written $(A, \{A_s, s \in A\})$ for short. «

To be well defined, the semantic functions in Definition 3.4 must satisfy four technical conditions.

Condition 3.6 (Normalized Prior)**.** The Prior semantic function induces a probability distribution over $(\mathcal{L}, \Sigma_{\mathcal{L}})$, in the sense that

$$\sum_{s \in S} \left[\int_{\theta \in \Theta_s} \text{Prior} \, [\![(s, \theta)]\!] \, \lambda_s(\mathrm{d}\theta) \right] = 1. \tag{3.3}$$

«

Condition 3.7 (Normalized Likelihood)**.** For each $t \in T$ and $E \in \mathcal{L}$, the function defined by the rule $x \mapsto \text{Likelihood} \, [\![E]\!] \, (t, x)$ induces a probability distribution over $(X_t, \mathcal{F}_t)$, in the sense that

$$\int_{x \in X_t} \text{Likelihood} \, [\![E]\!] \, (t, x) \; \lambda_t(\mathrm{d}x) = 1. \tag{3.4}$$

«

Condition 3.8 (Positive Likelihood)**.** For each $t \in T$ and $x \in X$, there exists a Prior positive measure set $A \subset \mathcal{L}$ such that for all $E \in A$, $\text{Likelihood} \, [\![E]\!] \, (t, x) > 0$. «

Condition 3.9 (Bounded Likelihood)**.** For each $t \in T$ and $x \in X$, the function defined by the rule $E \mapsto \text{Likelihood} \, [\![E]\!] \, (t, x)$ is measurable and essentially bounded, i.e., there exists a finite positive constant $c_{tx}^{\max}$ such that the set of expressions $\{E \in \mathcal{L} \mid \text{Likelihood} \, [\![E]\!] \, (t, x) > c_{tx}^{\max}\}$ is a subset of a Prior measure zero set. «

Remark 3.10. If Condition 3.6 holds, then the Prior probability of event $(A, \{A_s, s \in A\}) \in \Sigma_{\mathcal{L}}$ is

$$\sum_{s \in A} \left[\int_{\theta \in A_s} \text{Prior} \, [\![(s, \theta)]\!] \, \lambda_s(\mathrm{d}\theta) \right]. \tag{3.5}$$

«

The following proposition is a straightforward consequence of the fact that the expectation of a positive essentially bounded random variable lies between zero and its maximum value.

Proposition 3.11. *If Conditions 3.6–3.9 hold, then for each $t \in T$ and $x \in X$, the marginal likelihood c_{tx} is positive and finite, that is*

$$0 < c_{tx} ::= . \sum_{s \in S} \left[\int_{\theta \in \Theta_s} \text{Likelihood} \left[\!\left[(s, \theta) \right]\!\right] (t, x) \cdot \text{Prior} \left[\!\left[(s, \theta) \right]\!\right] \ \lambda_s(\mathrm{d}\theta) \right] < c_{tx}^{\max} < \infty. \tag{3.6}$$

«

When the semantic functions satisfy the above conditions, they induce a new semantic function $\text{Post} : \mathcal{L} \to \mathcal{X} \to \mathbb{R}_{\geq 0}$, called the posterior:

$$\text{Post} \left[\!\left[E \right]\!\right] (t, x) ::= \text{Likelihood} \left[\!\left[E \right]\!\right] (t, x) \cdot \text{Prior} \left[\!\left[E \right]\!\right] / c_{tx}. \tag{3.7}$$

Integrating Eq. (3.7) over $E = (s, \theta) \in \mathcal{L}$ establishes the following proposition.

Proposition 3.12. *If Conditions 3.6–3.9 hold, then for any observation $(t, x) \in \mathcal{X}$, the function defined by the rule $E \mapsto \text{Post} \left[\!\left[E \right]\!\right] (t, x)$ induces a probability distribution over $(\mathcal{L}, \Sigma_{\mathcal{L}})$, such that the probability of any event $(A, \{A_s, s \in A\}) \in \Sigma_{\mathcal{L}}$ is given by*

$$\sum_{s \in A} \left[\int_{\theta \in A_s} \text{Post} \left[\!\left[(s, \theta) \right]\!\right] (t, x) \lambda_s(\mathrm{d}\theta) \right]. \tag{3.8}$$

«

3.2 Bayesian Synthesis in Probabilistic DSLs

The Bayesian synthesis problem can now be stated.

Problem 3.13 (Bayesian Synthesis)**.** *Let D be a probabilistic domain-specific modeling language as in Definition 3.4, whose* Prior *and* Likelihood *semantics satisfy Conditions 3.6–3.9. Given an observation $O \in \mathcal{X}(D)$, generate samples of expressions $E \in \mathcal{L}$ from the posterior distribution over $(\mathcal{L}, \Sigma_{\mathcal{L}})$ defined by* $\text{Post} \left[\!\left[\cdot \right]\!\right] (O)$ *in Eq.* (3.7)*.* «

Figure 3.1 shows the workflow of Bayesian synthesis, which is next described in more detail.

3.2.1 Bayesian Synthesis via Sound Approximate Inference

In Figure 3.1, the input to Bayesian synthesis is a probabilistic DSL D and observation $O \in \mathcal{X}$. The goal is to synthesize an ensemble $\{E_1, \ldots, E_M\}$ of $M > 0$ DSL programs from the posterior Eq. (3.8), which collectively represent a set of likely structures and parameters that observation O. Because generating exact posterior samples is intractable in most settings, approximate sampling is used instead. The approximate sampling techniques described in Section 3.3 are *sound* in the sense that they produce DSL programs from a distribution that becomes arbitrarily close to the target distribution (3.8) given enough computational effort.

3.2.2 Querying Synthesized Probabilistic Programs

To discover structure in the observed data and make future predictions, the synthesized DSL programs are analyzed by solving *queries*. A query $\phi : \mathcal{L} \to \mathcal{Q}$ is a mapping from DSL programs to a space $\mathcal{Q}$ that defines the set of allowable results. Recalling that each expression $E = (s, \theta) \in \mathcal{L}$ represents the structure s and parameters θ for a family of statistical models, there are two classes of queries:

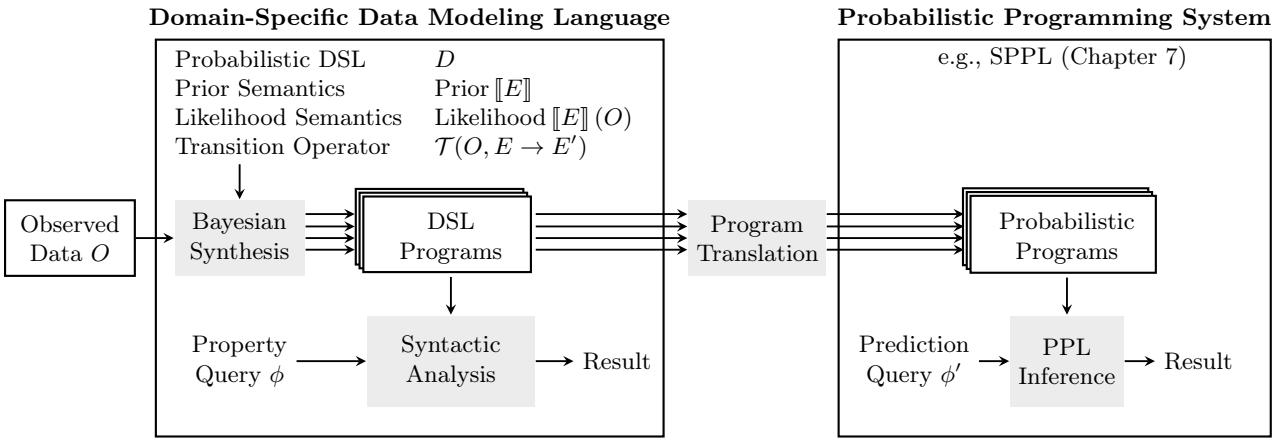

Figure 3.1: Components of Bayesian synthesis of probabilistic programs for automatic data modeling.

(i) *Property Queries.* This class of queries can be solved using simple syntactic analyses on the synthesized DSL expression to extract interpretable patterns. A predicate query has the signature $\phi : \mathcal{L} \to \{0, 1\}$, where $\phi\,[\![E]\!] = 1$ if E satisfies the predicate and 0 otherwise. Using the ensemble of M synthesized DSL expressions, an approximation to the posterior probability that $\phi\,[\![E]\!] = 1$ is given by

$$\Pr\{\phi\,[\![E]\!] = 1 \mid O\} \approx \frac{1}{M}\sum_{i=1}^{M} \phi\,[\![E_i]\!]\,. \tag{3.9}$$

The probabilities in Table 2.3 are solutions to property queries about the probability that a given temporal pattern is present.

(ii) *Prediction Queries.* This class of queries includes simulating new data or computing both marginal and conditional probabilities of events involving observations $(t, x) \in \mathcal{X}$. Rather than develop custom solvers for each DSL, prediction queries solved by first applying a syntactic operation Translate : $\mathcal{L} \to \mathcal{L}'$ that converts domain-specific programs $E \in \mathcal{L}$ into probabilistic programs $E' \in \mathcal{L}'$ in a probabilistic programming language. The Sum-Product Probabilistic Language (SPPL) described in Chapter 7 can serve as the unified target language $\mathcal{L}'$ for all probabilistic DSLs in Part I. Chapter 7 formalizes the SPPL modeling syntax and the class of queries that it can solve. Translating DSL expressions into SPPL programs enables substantial reuse of general-purpose inference machinery to automatically obtain exact results to queries, irrespective of the original DSL from which the SPPL program was translated. Using the ensemble of M synthesized DSL expressions, an approximation to the posterior probability that $\phi\,[\![E]\!] \in Q$ for some subset $Q \subset \mathcal{Q}$ is given by

$$\Pr\{\phi\,[\![E]\!] \in Q \mid O\} \approx \frac{1}{M}\sum_{i=1}^{M} \mathbf{1}\big[\phi'\,[\![\text{Translate}\,[\![E_i]\!]]\!] \in Q\big], \tag{3.10}$$

where ϕ' is the query in SPPL syntax. The 95% prediction intervals in the top row of Figure 2.5 are solutions to a prediction query.

Algorithm 3.1 Markov chain Monte Carlo algorithm for Bayesian synthesis.

Require: observation $O ::= (t, x) \in \mathcal{X}$, number of iterations $n \geq 1$

Ensure: sample from $\text{ApproxPost}^{(n)}_{E_0}[\![\cdot]\!](O)$ defined in Eqs. (3.15)–(3.16), for some $E_0 \in \mathcal{L}$ with Likelihood $[\![E_0]\!](O) > 0$

1: **procedure** BAYESIAN-SYNTHESIS-MCMC(O, n)
2: **do**
3: $E_0 \sim$ GENERATE-EXPRESSION-FROM-PRIOR() ▷ generate E_0 with probability Prior $[\![E]\!]$
4: **while** EVALUATE-LIKELIHOOD(O, E_0) $= 0$
5: **for** $i = 1 \ldots n$ **do** ▷ run n sampling iterations
6: $E_i \sim$ GENERATE-NEW-EXPRESSION(O, E_{i-1})
7: **return** $E_1, \ldots, E_n$

3.3 Algorithms for Bayesian Synthesis

Solving Problem 3.13 exactly is computationally intractable most settings. Sections 3.3.1 and 3.3.2 describe algorithm templates for randomly sampling expressions $E \in \mathcal{L}$ with probability that approximates Post $[\![E]\!](O)$ using Markov chain Monte Carlo (MCMC) and sequential Monte Carlo (SMC), respectively. These algorithms are sound in the sense that they are guaranteed converge to the posterior distribution in the limit of computation, thereby producing arbitrarily accurate approximations to queries using estimators of the form (3.9) and (3.10). Both algorithm templates require three procedures (P1)–(P3), shown below, for synthesis in a given probabilistic DSL. The procedure (P3), which transitions the current DSL expression, is often the main design challenge in engineering synthesis algorithms. Section 3.4 describes a generic and sound implementation of (P3) that applies to any probabilistic DSL generated by a context-free grammar.

(P1)	GENERATE-EXPRESSION-FROM-PRIOR()	sample $E \in \mathcal{L}$ with probability Prior $[\![E]\!]$
(P2)	EVALUATE-LIKELIHOOD(O, E)	evaluate Likelihood $[\![E]\!](O)$, for $E \in \mathcal{L}$, $O \in \mathcal{X}$
(P3)	GENERATE-NEW-EXPRESSION(O, E)	sample E' from E with probability $\mathcal{T}(O, E \to E')$

Remark 3.14. Let D be a probabilistic DSL. To avoid excessive measure-theoretic proofs, this section assumes that for each structure $s \in S$, the parameter space $\Theta_s = \{\theta_{s1}, \theta_{s2}, \ldots\}$ is countable, the sigma-algebra $\mathcal{F}_s = 2^{\Theta}_s$, and λ_s is the counting measure. As $\mathcal{L}(D)$ is now a countable union of countable sets, it is also countable. Eq. (3.5) is therefore written as a sum over $E \in \mathcal{E}$ for some $\mathcal{E} \subset \mathcal{L}$. «

3.3.1 Bayesian Synthesis via Markov Chain Monte Carlo

Algorithm 3.1 shows the template of an MCMC sampling algorithm for generating approximate solutions to Problem 3.13. In lines 2–4, the algorithm repeatedly calls GENERATE-EXPRESSION-FROM-PRIOR until it obtains an initial expression $E_0 \in \mathcal{L}$ such that Likelihood $[\![E]\!](O) > 0$. In lines 5–6, the algorithm iteratively invokes GENERATE-NEW-EXPRESSION to produce E_i from E_{i-1} using a *transition operator* $\mathcal{T}$. The transition operator takes as input expression E and observation O and stochastically samples an expression E' with probability denoted $\mathcal{T}(O, E \to E')$, where $\sum_{E' \in \mathcal{L}} \mathcal{T}(O, E \to E') = 1$ for all $E \in \mathcal{L}$. The next three conditions on the transition operator $\mathcal{T}$ are sufficient to prove that Algorithm 3.1 returns expressions $(E_1, \ldots, E_n)$ from an arbitrarily close approximation to Post $[\![E]\!](O)$.

Condition 3.15 (Posterior invariance)**.** If an expression $E \in \mathcal{L}$ is sampled from the posterior distribution and a new expression $E' \in \mathcal{L}$ is sampled with probability $\mathcal{T}(O, E \to E')$, then E' is also a sample from the posterior distribution:

$$\sum_{E \in \mathcal{L}} \text{Post}\left[\!\left[E\right]\!\right](O) \cdot \mathcal{T}(O, E \to E') = \text{Post}\left[\!\left[E'\right]\!\right](O). \tag{3.11}$$

«

Condition 3.16 (Posterior irreducibility)**.** Every expression $E' \in \mathcal{L}$ with nonzero likelihood is reachable from every $E \in \mathcal{L}$ in a finite number of steps. More specifically, for all pairs of expressions (E, E') for which $\text{Likelihood}\left[\!\left[E'\right]\!\right](O) > 0$ there exists an integer $n \geq 1$ and a sequence of expressions $E_1, E_2, \ldots, E_n$ where $E_1 = E$ and $E_n = E'$ such that $\mathcal{T}(O, E_{i-1} \to E_i) > 0$ for all $i \in \{2, \ldots, n\}$. «

Condition 3.17 (Aperiodicity)**.** There exists some expression $E \in \mathcal{L}$ such that the transition operator has a nonzero probability of returning to the same expression, i.e., $\mathcal{T}(O, E \to E) > 0$. «

It is next established that Algorithm 3.1 returns asymptotically sound samples whenever these three conditions hold. First, it is necessary to prove that an initial expression $E_0 \in \mathcal{L}$ for which $\text{Likelihood}\left[\!\left[E_0\right]\!\right](O) > 0$ can be obtained using a finite number of invocations of GENERATE-EXPRESSION-FROM-PRIOR.

Proposition 3.18. *The do-while loop of Algorithm 3.1 will terminate with probability 1, and the expected number of iterations is at most* $c_{tx}^{\max}/c_{tx}$. «

Proof. The number of iterations of the loop is geometrically distributed with mean

$$\begin{aligned} p &= \sum_{E \in \mathcal{L}} \text{Prior}\left[\!\left[E\right]\!\right] \cdot \mathbb{I}[\text{Likelihood}\left[\!\left[E\right]\!\right](O) > 0] && (3.12)\\ &= \frac{1}{c_{tx}^{\max}} \sum_{E \in \mathcal{L}} \text{Prior}\left[\!\left[E\right]\!\right] \cdot c_{tx}^{\max} \cdot \mathbb{I}[\text{Likelihood}\left[\!\left[E\right]\!\right](O) > 0] && (3.13)\\ &\geq \frac{1}{c_{tx}^{\max}} \sum_{E \in \mathcal{L}} \text{Prior}\left[\!\left[E\right]\!\right] \cdot \text{Likelihood}\left[\!\left[E\right]\!\right](O) = \frac{c_{tx}}{c_{tx}^{\max}}. && (3.14) \end{aligned}$$

Therefore, the expected number of iterations of the do-while loop is at most $1/p = c_{tx}^{\max}/c_{tx} < \infty$. ■

The notation $\mathcal{T}^n(O, E_0 \to (E_1, \ldots, E_n))$ denotes the probability that Algorithm 3.1 returns expressions $(E_1, \ldots, E_n)$ given $n \geq 1$ inputs, observation $O \in \mathcal{X}$, and initial expression E_0. For each $E \in \mathcal{L}$, the marginal probability that the return value of Algorithm 3.1 satisfies $E_n = E$ is denoted $\text{ApproxPost}_{E_0}^{(n)}\left[\!\left[E\right]\!\right](O)$ and can be defined inductively

$$\text{ApproxPost}_{E_0}^{(1)}\left[\!\left[E\right]\!\right](O) ::= \mathcal{T}(O, E_0 \to E) \tag{3.15}$$

$$\text{ApproxPost}_{E_0}^{(n)}\left[\!\left[E\right]\!\right](O) ::= \sum_{E' \in \mathcal{L}} \mathcal{T}(O, E' \to E) \cdot \text{ApproxPost}_{E_0}^{(n-1)}\left[\!\left[E'\right]\!\right](O) \qquad (n > 1). \tag{3.16}$$

The next two convergence theorems are due to Tierney [1994].

Theorem 3.19 (Convergence of MCMC [Tierney, 1994, Theorem 1])**.** *If Conditions 3.15–3.17 all hold for language* $\mathcal{L}$*, transition operator* $\mathcal{T}$*, and observation* $O \in \mathcal{X}$*, then for all* $E_0 \in \mathcal{L}$ *such that* $\text{Likelihood}\left[\!\left[E_0\right]\!\right](O) > 0$*,*

$$\sup_{\mathcal{E} \subset \mathcal{L}} \left| \sum_{E \in \mathcal{E}} \left[\text{ApproxPost}_{E_0}^{(n)}\left[\!\left[E\right]\!\right](O) - \text{Post}\left[\!\left[E\right]\!\right](O) \right] \right| \xrightarrow[n \to \infty]{} 0. \tag{3.17}$$

«

Eq. (3.17) is a very strong form of convergence known as convergence in total variation. It states that the maximum difference that the probability measures $\text{ApproxPost}_{E_0}^{(n)}[\![\cdot]\!](O)$ and $\text{Post}[\![\cdot]\!](O)$ assign to any set $\mathcal{E} \subset \mathcal{L}$ of expressions in the language can be made arbitrarily close to 0, for sufficiently large n.

Theorem 3.20 (Strong Law of Large Numbers for MCMC [Tierney, 1994, Theorem 3])**.** *If Conditions 3.15–3.17 hold for language $\mathcal{L}$, transition operator $\mathcal{T}$, and data $O \in \mathcal{X}$, then for all $E_0 \in \mathcal{L}$ with* $\text{Likelihood}[\![E_0]\!](O) > 0$ *and any real function $\phi : \mathcal{L} \to \mathbb{R}$ that satisfies $\sum_{E \in \mathcal{L}} |\phi(E)| \cdot \text{Post}[\![E]\!](O) < \infty$,*

$$\frac{1}{n}\sum_{i=1}^{n} \phi(E_i) \xrightarrow[n\to\infty]{} \sum_{E \in \mathcal{L}} \phi(E) \cdot \text{Post}[\![E]\!](O) =: \mathbb{E}_{\text{Post}}[\phi(E)] \qquad \textit{almost surely,} \tag{3.18}$$

where $(E_1, \ldots, E_n) \sim \mathcal{T}^n(E_0 \to \cdot)$. «

Eq. (3.18) states that the time average of $\phi(E_i)$ evaluated along a random sample path of the Markov chain converges with probability one to the expected value $\mathbb{E}_{\text{Post}}[\phi(E)]$ under the posterior.

Remark 3.21. It is common to run Algorithm 3.1 several times to obtain $M \geq 1$ independent MCMC chains $\{(E_1^j, \ldots, E_n^j)\}_{j=1}^M$. To estimate $\mathbb{E}[\phi(E)]$, the average-of-averages is computed

$$\frac{1}{m}\sum_{j=1}^{m}\left[\frac{1}{n}\sum_{i=B}^{n}\phi(E_i^j)\right], \tag{3.19}$$

where $B \in \{1, \ldots, n\}$ denotes a "burn-in" period. The heuristics of multiple chains and burn-in may improve the variance of the estimator and they do not impact the correctness of Theorem 3.20. «

3.3.2 Bayesian Synthesis via Resample-Move Sequential Monte Carlo

In many data modeling problems, the data has a natural interpretation as a stream of observations

$$(t_1, x_1), (t_2, x_2), (t_3, x_3), \ldots \tag{3.20}$$

where, using the notation from Definition 3.1, each $t_i \in T$ and $x_i \in X_{t_i}$ for $i \geq 1$. For example, in the Gaussian process DSL from Chapter 2, t_i is the list of all time points up to and including step i and x_i is the list of corresponding time series values. When the data has this kind of sequential structure, the same three primitives (P1)–(P3) can be used to design SMC synthesis algorithms that are more scalable than the MCMC approach in Algorithm 3.1.

Overview of SMC There are many algorithms that fall under the broad umbrella of "sequential Monte Carlo". The specific variant considered here is called "resample-move" SMC [Gilks and Berzuini, 2001]. Let $O_j ::= (t_j, x_j)$ be the j^{th} element in the sequence (3.20). The sequence of target distributions

$$\{\text{Post}[\![\cdot]\!](O_j)\}_{j=0}^{J} \tag{3.21}$$

are all defined over the same space $\mathcal{L}$ of DSL expressions[1]. For $j = 0$, the target distribution $\text{Post}[\![\cdot]\!](O_0)$ is simply $\text{Prior}[\![\cdot]\!]$. At each step $j = 1, \ldots, J$, the SMC algorithm maintains a set $\{(E_j^\ell, w_j^\ell)\}_{\ell=1}^M$ of $M \geq 1$ weighted expressions, where (informally) the weight w_j^ℓ represents how well the expression E_j^ℓ

[1]The setting where all the target distributions are defined on the same space is sometimes referred to as "static sequential Monte Carlo", to distinguish from the more common setting of state-space models where the dimensionality of the latent space increases at each step.

(S0) At step $j = 0$, initialize for each $\ell = 1, \ldots, M$

$$E_0^\ell \sim \textsc{Generate-Expression-From-Prior}(), \tag{3.23}$$

For iterations $j = 1, \ldots, J$, run steps (S1)–(S3):

(S1) **Reweight**: For $\ell = 1, \ldots, M$, incorporate observation O_j by recomputing the weights:

$$w_j^\ell \leftarrow \frac{\textsc{Evaluate-Likelihood}(O_j, E_{j-1}^\ell)}{\textsc{Evaluate-Likelihood}(O_{j-1}, E_{j-1}^\ell)}, \tag{3.24}$$

(S2) **Resample**: For $\ell = 1, \ldots, M$; if $j < J$ then resample the parents and reset the weights:

$$u \sim \text{Categorical}(w_j^1, \ldots, w_j^M), \quad E_j^\ell \leftarrow E_{j-1}^u \tag{3.25}$$

(S3) **Rejuvenate**: For $\ell = 1, \ldots, M$, rejuvenate E_j^ℓ times by running $n \geq 0$ iterations of MCMC:

$$E_j^\ell \sim \textsc{Generate-New-Expression}(O_j, E_j^\ell). \tag{3.26}$$

Listing 3.1: Resample-move sequential Monte Carlo for Bayesian synthesis.

explains O_j, relative to other particles. This set of weighted expressions forms a discrete "particle-based" approximation of the target distribution at step j,

$$\sum_{\ell=1}^{M} \frac{w_j^\ell}{\sum_{k=1}^{M} w_j^k} \delta_{E_j^\ell}(\cdot) \approx \text{Post}\,[\![\cdot]\!]\,(O_j), \tag{3.22}$$

where δ_E is an atomic probability measure at the expression $E \in \mathcal{L}$. Listing 3.1 explains how to sequentially construct such a particle approximation. In (S0), a set of M particles are initialized i.i.d. from the prior, as in line 3 of Algorithm 3.1. For $j = 1, \ldots, J$, the observations are incorporated, which involves: (S1) reweighting each particle by the ratio of likelihood of the full new data $X_{1:j}$ to the likelihood of the previous data $X_{1:j-1}$; (S2) resampling the particles based on their weights; (S3) rejuvenating the particles by running $n \geq 0$ iterations of $\mathcal{T}$, as in line 6 of Algorithm 3.1.

Remark 3.22. In the reweighting step (S1) of Listing 3.1, the ratio (3.24) is well defined across all executions of the SMC algorithm whenever, in addition to Conditions 3.7 and 3.9, the extra condition $\text{Likelihood}\,[\![E]\!]\,(O) > 0$ for all E and O holds. Otherwise the particle weights may collapse to zero and the algorithm as presented would fail. Del Moral et al. [2015] discuss alternative techniques for handling this type of particle collapse. «

Remark 3.23. In Listing 3.1, elements of the observation sequence $(O_1, \ldots, O_j)$ need not be incorporated one by one. Any batch of $B \in \{1, \ldots, J\}$ observations can be used. «

Remark 3.24. In the resampling step (S2) of Listing 3.1, a common heuristic is to use an adaptive resampling strategy rather than resampling at each iteration. Resampling is triggered at step $j \in \{1, \ldots, J\}$ if the effective sample size (a measure of the particle diversity) satisfies

$$\text{ESS}(w_j^{1:\ell}) ::= \frac{1}{\sum_{\ell=1}^{M} \left[(w_j^\ell / (\sum_{k=1}^{M} w_j^k))^2 \right]} < \text{ESS}_{\min} \in \{1, \ldots, M\}. \tag{3.27}$$

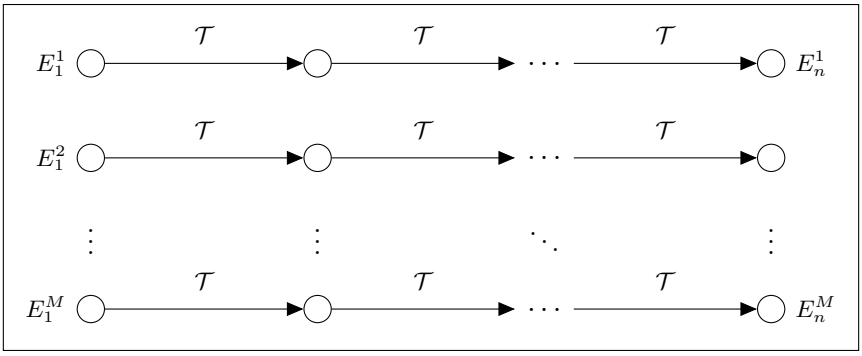

Figure 3.2: Markov chain Monte Carlo with M parallel chains executed for n iterations.

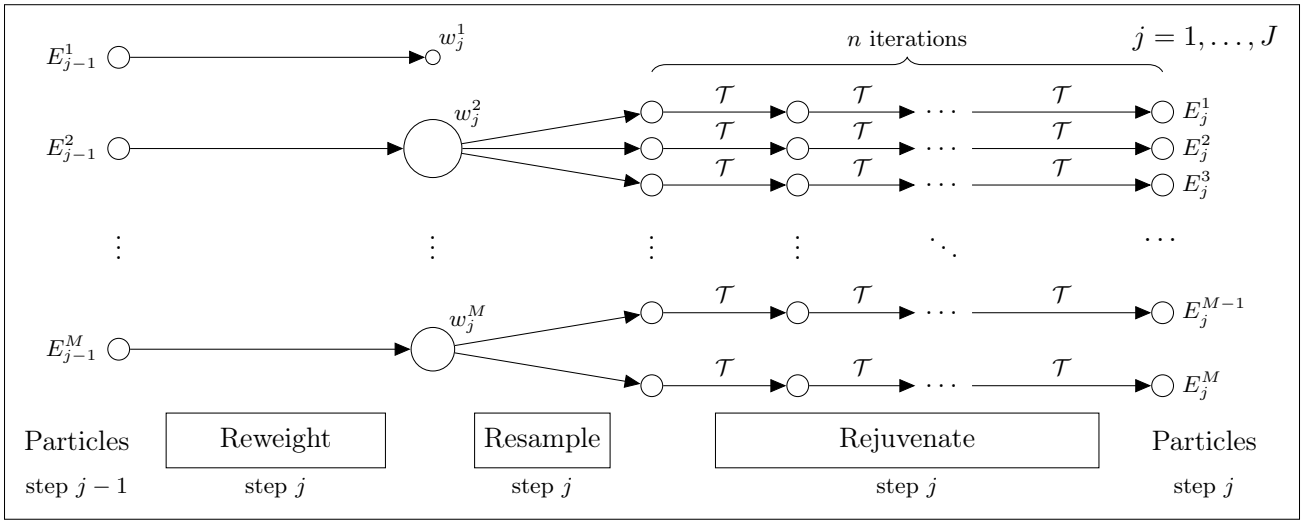

Figure 3.3: Resample-move sequential Monte Carlo with M particles and n rejuvenation iterations.

As with the MCMC heuristic from Remark 3.21, adaptive resampling does not impact correctness and may reduce variance at later steps. Resampling does not reduce variance at the present step, which is why resampling at the final step $j = J$ is avoided in (S2). Algorithm 3.2 shows an implementation of the resample-move SMC template in Listing 3.1 with adaptive resampling. «

Figures 3.2 and 3.3 show diagrams for MCMC and resample-move SMC, respectively. In situations where the observed data can be treated sequentially, SMC improves MCMC upon in the following ways:

- In MCMC with $M \geq 1$ parallel chains (Remark 3.21), each chain operates independently of the rest, whereas SMC with $M \geq 1$ particles redirects computational effort to more promising parts of the DSL $\mathcal{L}$ via resampling.
- MCMC does not easily apply to sequences of observations for online or streaming settings, as the target (posterior) distribution is fixed, whereas SMC handles streaming data through the sequence of posteriors (3.21).
- SMC delivers unbiased estimates of the marginal likelihood (3.29) of the observed data, which have many uses; for example, performing model comparison among a collection set of DSLs $\{\mathcal{L}_1, \ldots, \mathcal{L}_T\}$ that operate over the same data space $\mathcal{X}$.

Algorithm 3.2 Resample-move sequential Monte Carlo algorithm for Bayesian synthesis.

Require: observations $(O_1, \ldots, O_J)$ where $O_i \in \mathcal{X}$; number of move iterations $n \geq 0$; number of particles $M \geq 1$; $\mathrm{ESS}_{\min} \in \{1, \ldots, M\}$

procedure BAYESIAN-SYNTHESIS-SMC(X, n, M)
  for $\ell = 1 \ldots M$ **do** ▷ initialize M particles
    $E_0^\ell \sim$ GENERATE-EXPRESSION-FROM-PRIOR()
    $w_0^\ell \leftarrow 1; \quad \hat{w}_{j-1}^\ell \leftarrow 1$
  for $j = 1 \ldots J$ **do** ▷ for each observation in the sequence
    `// REWEIGHT`
    for $\ell = 1 \ldots M$ **do**
      $$w_j^\ell \leftarrow \hat{w}_{j-1}^\ell \cdot \frac{\text{EVALUATE-LIKELIHOOD}(O_j, E_{j-1}^\ell)}{\text{EVALUATE-LIKELIHOOD}(O_{j-1}, E_{j-1}^\ell)}$$
    `// RESAMPLE (ADAPTIVE)`
    if $\mathrm{ESS}(w_{j-1}^{1:M}) < \mathrm{ESS}_{\min}$ and $j < J$ **then** ▷ resampling triggered
      for $\ell = 1 \ldots M$ **do** ▷ resample particle
        $A_j^\ell \leftarrow \mathrm{Categorical}(M; w_j^{1:M})$ ▷ reset weight
        $\hat{w}_j^\ell \leftarrow 1$
    else
      for $\ell = 1 \ldots M$ **do** ▷ no resampling triggered
        $A_i^\ell \leftarrow \ell$ ▷ copy particle
        $\hat{w}_j^\ell \leftarrow w_{j-1}^\ell$ ▷ reset weight
    `// REJUVENATE`
    for $\ell = 1 \ldots M$ **do**
      $E_j^\ell \leftarrow E_{j-1}^{A_j^\ell}$ ▷ set parent to new ancestor
      for $i = 1 \ldots n$ **do**
        $E_j^\ell \sim$ GENERATE-NEW-EXPRESSION(O_j, E_j^ℓ) ▷ run transition operator
  return $E_{0:J}^{1:M}, w_{0:J}^{1:M}, A_{1:J}^{1:M}$

Convergence Properties If the transition operator $\mathcal{T}(O, E \to E')$, which is invoked by calling GENERATE-NEW-EXPRESSION in (S3), satisfies Conditions 3.15–3.17 for each target distribution in the sequence (3.21), then estimates of expectations $\mathbb{E}_{\mathrm{Post}}[\phi(E)]$ using the particle-based approximation (3.22) have a similar consistency property to Theorem 3.20, albeit for a more restricted class of functions ϕ. That is for each $j = 1, \ldots, J$,

$$\frac{1}{M} \sum_{\ell=1}^{M} W_j^\ell \phi(E_j^\ell) \xrightarrow[M \to \infty]{} \sum_{E \in \mathcal{L}} \phi(E) \cdot \mathrm{Post}\,[\![E]\!]\,(O_j) \qquad \text{almost surely.} \tag{3.28}$$

Precise statements about a central limit theorem and technical conditions on ϕ can be found in Gilks and Berzuini [2001, Theorem 1], Chopin [2004, Theorem 1], and Del Moral et al. [2006, Proposition 2]. Unlike MCMC, however, SMC has the remarkable property that it also delivers unbiased estimates of the marginal likelihood of each O_j $(j = 1, \ldots, J)$. Using the notation from Algorithm 3.2,

$$\mathbb{E}\left[\prod_{j=1}^{J}\left[\frac{1}{\sum_{\ell=1}^{M} \hat{w}_{j-1}^\ell} \sum_{\ell=1}^{M} w_j^\ell\right]\right] = \sum_{E \in \mathcal{L}} \mathrm{Likelihood}\,[\![E]\!]\,(O_j) \cdot \mathrm{Prior}\,[\![E]\!]\,. \tag{3.29}$$

3.4 Bayesian Synthesis for Context-Free Probabilistic DSLs

The preceding sections have introduced probabilistic DSLs (Section 3.1), formalized the Bayesian synthesis problem (Section 3.2), and presented MCMC and SMC algorithm templates (Section 3.3) that deliver sound approximate solutions to Bayesian synthesis under well characterized technical conditions. This section introduces a new class of context-free grammars that are used to define "context-free" probabilistic DSLs and outlines a provably sound implementation of the end-to-end Bayesian synthesis framework that applies to any context-free probabilistic DSL.

- Sections 3.4.1 and 3.4.2 presents a class of context-free grammars for specifying probabilistic DSLs.
- Section 3.4.3 shows how to implement (P1) GENERATE-EXPRESSION-FROM-PRIOR for context-free probabilistic DSLs by defining a Prior semantics that is guaranteed to be normalized as in Condition 3.6.
- Section 3.4.4 shows how to implement (P3) GENERATE-NEW-EXPRESSION for any context-free probabilistic DSL and proves that the transition operator satisfies Conditions 3.15–3.17 which ensure convergence of the MCMC and SMC algorithms in Section 3.3.

Remark 3.25. The procedure (P2) EVALUATE-LIKELIHOOD depends on the modeling application (e.g., Eq. (3.63))—any semantics that satisfies Conditions 3.7 and 3.9 is sufficient. «

3.4.1 Context-Free Grammars for Specifying Probabilistic DSLs

Formal grammars describe a recipe for how to form strings of a language starting from an alphabet of terminal symbols [Jelinek et al., 1992]. This section describes a new class of context-free grammars for defining probabilistic domain-specific data modeling languages whose expressions resemble Lisp-style symbolic expressions, or "s-expressions". These context-free grammars have two main properties:

(i) The grammar produces s-expressions that contain a unique phrase tag for each production rule, which uniquely identifies the production rule that produced a given subexpression;

(ii) Each production rule is associated with a probability distribution over terminal symbols that are jointly sampled to fill a finite number of "holes" in the production rule.

The first property guarantees that the grammar is unambiguous, i.e., each expression has a unique parse, which makes it straightforward to compute the probability of generating a given expression without enumerating over all possible parses. The second property is used to model random parameters.

Definition 3.26. A *context-free grammar for a probabilistic DSL* consists of six entities.

- $N ::= \{N_1, \ldots, N_m\}$ is a finite set of non-terminal symbols.
- $R ::= \{r_{ik} \mid i = 1, \ldots, m; k = 1, \ldots, r_i\}$ is a set of production rules, where r_{ik} is the k^{th} production rule of non-terminal N_i. Each production rule r_{ik} is a tuple of the form

$$r_{ik} ::= (N_i, t_{ik}, h_{ik}, \widetilde{N}_{ik}^1, \ldots, \widetilde{N}_{ik}^{n_{ik}}), \tag{3.30}$$

  where h_{ik} is the number of holes in the production; n_{ik} the number of non-terminals; and $\widetilde{N}_{ik}^j \in N$ for $j = 1, \ldots, n_{ik}$. If $n_{ik} = 0$ then r_{ik} is a *non-recursive* production rule, otherwise it is *recursive*.
- $T ::= \{t_{ik} \mid i = 1, \ldots, m; k = 1, \ldots, r_i\}$ is a set of phrase tag symbols, disjoint from N, where t_{ik} is a unique symbol that identifies the production rule r_{ik}.

- $P ::= \{p_{ik} \mid i = 1, \ldots, m; k = 1, \ldots, r_i\}$ is a set of probabilities, where each phrase tag t_{ik} is assigned a probability $p_{ik} \in (0, 1]$ that non-terminal N_i selects production rule r_{ik}.
- $W ::= \{(\Theta_{ik}, \Sigma_{ik}, \mu_{ik}) \mid i = 1, \ldots, m; k = 1, \ldots, r_i; h_{ik} > 0\}$ is a family of probability spaces, which specify a joint distribution over the allowable valuables that the h_{ik} holes in the production rule r_{ik} may take.
- $N^{\text{start}} \in N$ is a designated start symbol.

«

Example 3.27. Consider the Gaussian process DSL in Listing 2.1 from Chapter 2. Listing 3.2 shows a context-free grammar that more formally defines this DSL. The start symbol $N^{\text{start}} = N_1$ and the phrase tags $T = \{t_{11}, t_{21}, t_{31}, \ldots, t_{37}\}$ are shown in teletype font. The notation $B(U)$ denotes the Borel sigma-algebra of a topological space U, which is generated by the open sets. «

3.4.2 Defining a Probabilistic DSL from a Context-Free Grammar

This section shows how a context-free grammar G (Definition 3.26) defines the structure space S and measure spaces $(\Theta_s, \mathcal{F}_s, \lambda_s)$ (for each $s \in S$) of a probabilistic DSL D (Definition 3.4).

Eq. (3.30) describes how the production rules of G convert non-terminal symbols into tagged s-expressions with holes. The evaluation $N_i \Downarrow_G^p E$ means that, starting from non-terminal N_i, the s-expression E is yielded with probability p. Recalling that both h_{ik} and n_{ik} in Eq. (3.30) can be zero, the following rule applies for all $i = 1, \ldots, n$ and $k = 1, \ldots, r_i$:

$$\frac{(N_i, t_{ik}, h_{ik}, \widetilde{N}_{ik}^1, \ldots, \widetilde{N}_{ik}^{n_{ik}}) \in R \quad \widetilde{N}_{ik}^1 \Downarrow_G^{p_1} E_1 \quad \ldots \quad \widetilde{N}_{ik}^{n_{ik}} \Downarrow_G^{p_{n_{ik}}} E_{n_{ik}}}{N_i \Downarrow_G^{p_{ik} \prod_{j=1}^{n_{ik}} p_j} (t_{ik}\ \square_1\ \ldots\ \square_{h_{ik}}\ E_1\ \ldots\ E_{n_{ik}})} \tag{3.31}$$

Remark 3.28. The rule (3.31) always yields an s-expression where the first element is a phrase tag t_{ik} that unambiguously identifies the production rule r_{ik} that yielded the expression. Thus, every s-expression has a unique parse tree [Turbak and Gifford, 2008, Section 2.3.3]. «

The language $\mathcal{L}_\square(G, N_i)$ denotes the set of all s-expressions with holes that can be yielded starting from non-terminal N_i $(i = 1, \ldots, m)$ with positive probability according to the relation defined by Eq. (3.31). Further, the language $\mathcal{L}_\square(G) ::= \mathcal{L}_\square(G, N^{\text{start}})$ is the set of all s-expressions derivable from the start symbol. With these notations, the structure space S and probabilities p_s $(s \in S)$ are

$$S ::= \mathcal{L}_\square(G) \tag{3.32}$$

$$p_s ::= w \in (0, 1] \text{ such that } N^{\text{start}} \Downarrow_G^w s. \tag{3.33}$$

Each probability p_s is unique by Remark 3.28. Next, the probability spaces $\{(\Theta_s, \mathcal{F}_s, \mu_s), s \in S\}$ of D are obtained from a transition relation $s \Downarrow_G (\Theta_s, \Sigma_s, \mu_s)$, defined as follows:

$$\frac{}{(t_{ik}\ \square_1\ \ldots\ \square_{h_{ik}}) \Downarrow_G (\Theta_{ik}, \Sigma_{ik}, \mu_{ik})} \qquad \text{(Primitive-Production)}$$

$$\frac{s_1 \Downarrow_G^\Theta (\Theta_1, \mathcal{F}_1, \mu_1) \quad \ldots \quad s_{n_{ik}} \Downarrow_G^\Theta (\Theta_{n_{ik}}, \mathcal{F}_{n_{ik}}, \mu_{n_{ik}})}{(t_{ik}\ \square_1\ \ldots\ \square_{h_{ik}}\ s_1 \ldots\ s_{n_{ik}}) \Downarrow_G (\prod_{j=1}^{n_{ik}} \Theta_j, \otimes_{j=1}^{n_{ik}} \mathcal{F}_j, \times_{j=1}^{n_{ik}} \mu_j)} \qquad \text{(Recursive-Production)}$$

where

$$\prod_{j=1}^{n_{ik}} \Theta_j ::= \{(\theta_1, \ldots, \theta_{n_{ik}}) \mid \theta_j \in \Theta_j, j = 1, \ldots n_{ik}\} \qquad \text{(Cartesian product)} \tag{3.34}$$

$$\otimes_{j=1}^{n_{ik}} \mathcal{F}_j ::= \sigma\left(\left\{\prod_{j=1}^{n_{ik}} A_j \mid A_j \in \mathcal{F}_j, j = 1, \ldots, n_{ik}\right\}\right) \qquad \text{(product sigma-algebra)} \tag{3.35}$$

$$(\times_{j=1}^{n_{ik}} \mu_j)\left(\prod_{j=1}^{n_{ik}} A_j\right) ::= \prod_{j=1}^{n_{ik}} \mu_j(A_j),\ A_j \in \mathcal{F}_j, j = 1, \ldots, n_{ik} \qquad \text{(product measure).} \tag{3.36}$$

Remark 3.29. By Billingsley [1995, Theorem 18.2], the product measure (3.36) defined on the generating sets extends to a unique probability measure on the entire sigma-algebra (3.35). «

Remark 3.30. The context-free property of G means that the joint probability distributions for holes that belong to different subexpressions are obtained in Eq. (RECURSIVE-PRODUCTION) by taking product measures. While the h_{ik} holes in a s-expression with phrase tag t_{ik} have an arbitrary joint distribution μ_{ik}, the holes in different subexpressions are mutually independent. «

Eqs. (3.32)–(3.36) how a context-free grammar G uniquely defines the set of expressions (3.1) of a probabilistic DSL D.

Definition 3.31. Let G be a context-free grammar and D a probabilistic DSL whose expressions match those generated by G. For each $(s, \theta) \in \mathcal{L}(D)$, the notation $s \oplus \theta$ denotes the unique s-expression obtained by syntactically replacing each hole in s with the corresponding parameter in θ. The languages

$$\mathcal{L}(G, N_i) ::= \{s \oplus \theta \mid s \in \mathcal{L}_\square(G, N_i), \theta \in \Theta_s\} \qquad (i = 1, \ldots, m) \tag{3.37}$$

are defined to be the set of all s-expressions that can be yielded starting from the non-terminal N_i and where the holes are replaced with valid parameter values. Moreover, the language $\mathcal{L}(G) ::= \mathcal{L}(G, N^{\text{start}})$. «

Example 3.32. Let G be the grammar in Listing 3.2 and D a probabilistic DSL whose expressions match those generated by G. Consider the following structure $s \in \mathcal{L}_\square(G)$ and parameter $\theta \in \Theta_s$:

$$s ::= \texttt{(GaussianProcess (Noise □) (* (Constant □) (Periodic □ □)))} \tag{3.38}$$

$$\theta ::= ((), (0.7, (1.8, (2.1, 3.1)))). \tag{3.39}$$

As $(s, \theta) \in \mathcal{L}(D)$, applying Definition 3.31 gives

$$s \oplus \theta = \texttt{(GaussianProcess (Noise 0.7) (* (Constant 1.8) (Periodic 2.1 3.1)))}.$$

The formal definition of $s \oplus \theta$ is straightforward. The uniqueness of $s \oplus \theta$ follows from Remark 3.28. «

Remark 3.33. As the languages $\mathcal{L}(G)$ and $\mathcal{L}(D)$ are in 1-1 correspondence, the expressions (s, θ) and $s \oplus \theta$ will be used interchangeably, whichever is clearer or more notationally compact. Any semantic function defined on one of these domains is therefore also defined on the other. «

3.4.3 A Sound Prior Semantics

Consider a context-free grammar G and the corresponding language $\mathcal{L}_\square(G)$ of all derivable s-expressions with holes. To meet Condition 3.6, setting $\text{Prior}\,[\![(s,\theta)]\!]\,\lambda_s(\mathrm{d}\theta) = p_s\mu_s(\mathrm{d}\theta)$ is sufficient since

$$\sum_{s\in S}\left[\int_{\theta\in\Theta_s}\text{Prior}\,[\![(s,\theta)]\!]\,\mu_s(\mathrm{d}\theta)\right] = \sum_{s\in S} p_s\mu(\Theta_s) = 1. \tag{3.40}$$

Eq. (3.40) is well formed if and only if the structure probabilities $\{p_s \mid s \in \mathcal{L}_\square(G)\}$ defined in Eq. (3.33) sum to one. A classic result from Booth and Thompson [1973] can be used to verify this property.

Definition 3.34 (Consistency [Booth and Thompson, 1973])**.** A context-free grammar G is consistent if the probabilities assigned to all strings derivable from G sum to one. «

Theorem 3.35 (Sufficient Condition for Consistency [Booth and Thompson, 1973])**.** *A proper probabilistic context-free grammar G is consistent if the largest eigenvalue (in modulus) of the expectation matrix of G is less than one.* «

Remark 3.36. Gecse and Kovács [2010] show how to construct the expectation matrix of a context-free grammar G from the production rule probabilities $\{p_{ik}\}$. If the production rules enforce finite recursion depth, then G is consistent since $\mathcal{L}_\square(G)$ is finite. If the production rules allow arbitrary recursion depth, then consistency of G is equivalent to having a finite expected number of steps in the rewriting rule Eq. (3.31). Every context-free grammar is henceforth assumed to be consistent. «

In most modeling applications, the distributions μ_{ik} for the holes in the context-free grammar G have a density γ_{ik} with respect to a sigma-finite dominating measure λ_{ik} (e.g., some combination of Lebesgue and counting measures) over Σ_{ik}. For $s \in \mathcal{L}_\square(G)$, the measure λ_s is defined to be the product the measure over $(\Omega_s, \mathcal{F}_s)$ analogously to Eq. (3.36) and $\gamma_s : \Theta_s \to \mathbb{R}_{\geq 0}$ denotes the product densities of μ_s. A default denotational semantics $\text{Prior} : \mathcal{L}(G) \to \mathbb{R}_{>0}$ can now be described for this class of applications. To aid with the construction, the semantic function $\text{Expand} : \mathcal{L}(G) \to N \to \mathbb{R}_{\geq 0}$ is used to map an expression and a non-terminal symbol to the probability density that the non-terminal is expanded to the given expression:

$$\text{Expand}\,[\![(t_{ik}\ \theta_1\ \ldots\ \theta_{h_{ik}}\ E_1\ \ldots\ E_{n_{ik}}\ )]\!]\,(N_i) ::= p_{ik}\prod_{j=1}^{h_{ik}}\gamma_{ik}(\theta_1,\ldots,\theta_{h_{ik}})\prod_{j=1}^{n_{ik}}\text{Expand}\,[\![E_j]\!]\,(\widetilde{N}_{ik}^{j}) \tag{3.41}$$

for $i = 1,\ldots,n$ and $k = 1,\ldots,r_i$. The prior probability of an expression E is then

$$\text{Prior}\,[\![E]\!] ::= \text{Expand}\,[\![E]\!]\,(N^{\text{start}}). \tag{3.42}$$

Proposition 3.37. *The prior density of $E = (s,\theta)$ in Eq. (3.42) factorizes as*

$$\text{Prior}\,[\![(s,\theta)]\!] = p_s\gamma_s(\theta). \tag{3.43}$$

«

Proof. Combine Eq. (3.31) and Eq. (3.41) and perform structural induction on $(s,\theta) \in \mathcal{L}$. ■

Remark 3.38. The context-free structure of G reflects two types of probabilistic independencies. First, the joint probability of $(s,\theta) \in \mathcal{L}(D)$ factorizes according to $p_s\gamma_s(\theta)$, which means that the entire structure is sampled independently of the numeric parameters. Second, the parameters $(\theta_1,\ldots,\theta_{h_{ik}})$ in any subexpression $(t_{ik}\ \theta_1\ \ldots\ \theta_{h_{ik}}\ E_1\ \ldots\ E_{n_{ik}}\ )$ are independent of the parameters that appear in $E_1,\ldots,E_{n_{ik}}$, which are themselves mutually independent of one another. «

Algorithm 3.3 Transition operator $\mathcal{T}$ for a context-free language.

procedure GENERATE-NEW-EXPRESSION(O, E) ▷ observation $O \in \mathcal{X}$ and input expression $E \in \mathcal{L}$
  $a \sim \text{Uniform}(A_E)$ ▷ randomly select a node in parse tree
  $(N_i, E_{\text{sev}}) \leftarrow \text{Sever}_a [\![E]\!]$ ▷ sever parse tree and return non-terminal symbol at sever point
  $E_{\text{sub}} \sim \text{Expand} [\![\cdot]\!] (N_i)$ ▷ generate random E_{sub} with probability $\text{Expand} [\![E_{\text{sub}}]\!] (N_i)$
  $E' \leftarrow E_{\text{sev}}[E_{\text{sub}}]$ ▷ fill hole in E_{sev} with expression E_{sub}
  $L \leftarrow \text{Likelihood} [\![E]\!] (O)$ ▷ evaluate likelihood for expression E and observation O
  $L' \leftarrow \text{Likelihood} [\![E']\!] (O)$ ▷ evaluate likelihood for expression E' and observation O
  $p_{\text{accept}} \leftarrow \min \{1, (|A_E|/|A_{E'}|) \cdot (L'/L)\}$ ▷ compute the probability of accepting the mutation
  $r \sim \text{Uniform}([0, 1])$ ▷ draw a random number from the unit interval
  if $r < p_{\text{accept}}$ **then** ▷ if-branch has probability p_{accept}
    return E' ▷ accept and return the mutated expression
  else ▷ else-branch has probability $1 - p_{\text{accept}}$
    return E ▷ reject the mutated expression and return the input expression

The notation $E \sim \text{Expand} [\![\cdot]\!] (N_i)$ is used to mean that E is randomly sampled according to the density defined in Eq. (3.41). That is, to expand N_i, a production rule r_{ik} is chosen with probability p_{ik}, the parameters $(\theta_1, \ldots, \theta_{h_{ik}})$ are sampled jointly from μ_s to fill in the holes, and the subexpressions (if any) for $\widetilde{N}_{ik}^j$ $(j = 1, \ldots, n_{ik})$ are sampled recursively. (P1) GENERATE-EXPRESSION-FROM-PRIOR() returns $E \sim \text{Expand} [\![\cdot]\!] (N^{\text{start}})$, which is guarnateed to halt whenever G is consistent.

3.4.4 A Sound Markov Chain Transition Operator

Remark 3.39. This section makes the countability assumption from Remark 3.14. Following Remark 3.33, it is also assumed that each expression $(s, \theta) \in \mathcal{L}(D)$ is written in its unique s-expression form $(s \oplus \theta) \in \mathcal{L}(G)$, where the holes in s are syntactically replaced with parameters θ. «

This section defines a generic implementation of (P3) GENERATE-NEW-EXPRESSION for any language $\mathcal{L} ::= \mathcal{L}(G)$ whose structure and parameter spaces are defined by a context-free grammar G and proves that the implementation satisfies Conditions 3.15–3.17. Algorithm 3.3 shows the transition operator. Given an initial expression $E \in \mathcal{L}$, a randomly selected subexpression in E is replaced to obtain a new expression E', which is accepted according to the usual Metropolis-Hastings rule. The notation $\mathcal{T}(O, E \to E')$ denotes the probability that GENERATE-NEW-EXPRESSION(O, E) returns E'. Before proving correctness, a scheme for uniquely identifying syntactic locations in the parse tree of expressions is described.

Definition 3.40. Let $A ::= \{(a_1, a_2, \ldots, a_l) \mid a_i \in \{1, 2, \ldots, h_{\max}\}, l \in \{0, 1, 2, \ldots\}\}$ be a countably infinite set whose elements index nodes in parse trees of s-expressions. The integer $h_{\max}$ denotes the maximum number of symbols that appear on the right of any production rule (3.30). Each $a \in A$ is a sequence of subexpression positions on the path from the root node of a parse tree to another node. Further, the set $A_E \subset A$ denotes the finite subset of nodes that exist in the parse tree of $E \in \mathcal{L}$. «

Example 3.41. Suppose that $E = (t_0\ E_1\ E_2)$ where $E_1 = (t_1\ E_3\ E_4)$ and $E_2 = (t_2\ E_5\ E_6)$. The root node of the parse tree has index $a_{\text{root}} ::= ()$; the node for E_1 has index (1); the node for E_2 has index (2); the nodes for E_3 and E_4 have indices $(1, 1)$ and $(1, 2)$, respectively; and the nodes for E_5 and E_6 have indices $(2, 1)$ and $(2, 2)$, respectively. Thus, $A_E = \{(), (1), (2), (1, 1), (1, 2), (2, 1), (2, 2)\}$. «

Definition 3.42. The syntactic operation Sever, parametrized by $a \in A$, takes as input an expression E. If E contains a node a, then Sever returns a tuple (N_i, E_{sev}) where E_{sev} is the expression with the

subexpression of E located at a replaced with a symbol $\triangle$ and where N_i is the non-terminal symbol from which the removed subexpression is produced. Otherwise, Sever fails. Define the relation $\xrightarrow[\text{sever}]{}$

$$\frac{a = ()}{(a, (T_{ik}\ \theta_1\ \ldots\ \theta_{h_{ik}}\ E_1\ \ldots\ E_{n_{ik}})) \xrightarrow[\text{sever}]{} (N_i, \triangle)} \tag{3.44}$$

$$\frac{((a_2, a_3, \ldots), E_j) \xrightarrow[\text{sever}]{} (N_i, E_{\text{sev}}) \text{ and } a_1 = j}{\begin{aligned}(a, (T_{ik}\ \theta_1\ \ldots\ \theta_{h_{ik}}\ E_1\ \ldots\ E_{n_{ik}})) \xrightarrow[\text{sever}]{} (N_i, (T_{ik}\ \theta_1\ \ldots\ \theta_{h_{ik}} \\ E_1\ \ldots\ E_{j-1}\ E_{\text{sev}}\ E_{j+1}\ \ldots\ E_{n_{ik}}))\end{aligned}} \tag{3.45}$$

for $i = 1, \ldots, m$, $k = 1, \ldots, r_i$ and put

$$\text{Sever}_a [\![(T_{ik}\ \theta_1\ \ldots\ \theta_{h_{ik}}\ E_1\ \ldots\ E_{n_{ik}})]\!] ::= \begin{cases} (N_i, E_{\text{sev}}) & \text{if } (a, (T_{ik}\ \theta_1\ \ldots\ \theta_{h_{ik}}\ E_1\ \ldots\ E_{n_{ik}})) \\ & \quad \xrightarrow[\text{sever}]{} (N_i, E_{\text{sev}}) \\ \textbf{undefined} & \text{otherwise.} \end{cases} \tag{3.46}$$

«

Remark 3.43. For any expression $E \in \mathcal{L}$, setting $a = a_{\text{root}} \equiv ()$ gives $((), E) \xrightarrow[\text{sever}]{} (N^{\text{start}}, \triangle)$. Further, every $E_{\text{sev}} \notin \mathcal{L}$ because E_{sev} contains one or more instances of the special symbol $\triangle$. «

Remark 3.44. To handle s-expressions that contain a $\triangle$ subexpression, Eq. (3.41) is extended with the rule $\text{Expand}\,[\![\triangle]\!]\,(N_i) ::= 1$ for $i = 1, \ldots, m$. «

Definition 3.45. Let E_{sev} be an expression that contains a single $\triangle$ for which $(a, E) \xrightarrow[\text{sever}]{} (N_i, E_{\text{sev}})$ for some $E \in \mathcal{L}$, $a \in A$, and $i \in [m]$. The replacement $E_{\text{sev}}[E_{\text{sub}}] \in \mathcal{L}$ denotes the expression formed by replacing $\triangle$ with E_{sub}, where $E_{\text{sub}} \in \mathcal{L}(G, N_i)$ «

Definition 3.46. The operation Subexpr, parametrized by a, takes an expression E and extracts the subexpression corresponding to node a in the parse tree. That is, for $i = 1, \ldots, m$ and $k = 1, \ldots, r_i$,

$$\text{Subexpr}_a [\![(T_{ik}\ \theta_1\ \ldots\ \theta_{h_{ik}}\ E_1\ E_2\ \ldots\ E_{n_{ik}})]\!] ::= \begin{cases} \varnothing & \text{if } a = () \text{ or } a_1 > l \\ E_j & \text{if } a = (j) \text{ for some } 1 \le j \le l \\ \text{Subexpr}_{(a_2, a_3, \ldots)} [\![E_j]\!] & \text{if } a \ne (j) \text{ and} \\ & \quad a_1 = j \text{ for some } 1 \le j \le l. \end{cases}$$

«

Consider the probability that $\mathcal{T}$ takes an expression E to another expression E', which by total probability is an average over the uniformly chosen node index a:

$$\mathcal{T}(O, E \to E') = \frac{1}{|A_E|} \sum_{a \in A_E} \mathcal{T}(O, E \to E'; a) = \frac{1}{|A_E|} \sum_{a \in A_E \cap A_{E'}} \mathcal{T}(O, E \to E'; a), \tag{3.47}$$

where

$$\mathcal{T}(O, E \to E'; a) ::= \begin{cases} \text{Expand}\,[\![\text{Subexpr}_a [\![E']\!]]\!]\,(N_i) \cdot \alpha(E, E') & \text{if } \text{Sever}_a [\![E]\!] = \text{Sever}_a [\![E']\!] \\ & \quad = (N_i, E_{\text{sev}}) \text{ for some } i \text{ and } E_{\text{sev}}, \\ 0 \qquad \text{otherwise} \end{cases} \tag{3.48}$$

$$\alpha(E, E') ::= \min \left\{ 1, \frac{|A_E| \cdot \text{Likelihood}\,[\![E']\!]\,(O)}{|A_{E'}| \cdot \text{Likelihood}\,[\![E]\!]\,(O)} \right\}.$$

The second equality in Eq. (3.47) discards terms $a \in A_E \setminus A_{E'}$ from the sum, as these terms have $\text{Sever}_a [\![E']\!] = \varnothing$ and $\mathcal{T}(O, E \to E'; a) = 0$.

Proposition 3.47. *For $E \in \mathcal{L}$, if $\text{Sever}_a [\![E]\!] = (N_i, E_{\text{sev}})$ then $\text{Expand} [\![\text{Subexpr}_a [\![E]\!]]\!] (N_i) > 0$.* «

Proof. If $\text{Sever}_a [\![E]\!] = (N_i, E_{\text{sev}})$ then $\text{Subexpr}_a [\![E]\!]$ is an expression with tag t_{ik} for some k. Since $E \in \mathcal{L}$, it must be that $\text{Expand} [\![\text{Subexpr}_a [\![E]\!]]\!] (N_i) > 0$, as otherwise $\text{Prior} [\![E]\!] = 0$ (a contradiction). ■

Proposition 3.48. *For $E \in \mathcal{L}$, if $\text{Sever}_a [\![E]\!] = (N_i, E_{\text{sev}})$ then:*

$$\text{Expand} [\![E]\!] (S) = \text{Expand} [\![\text{Subexpr}_a [\![E]\!]]\!] (N_i) \cdot \text{Expand} [\![E_{\text{sev}}]\!] (S). \tag{3.49}$$

«

Proof. Each factor in $\text{Expand} [\![E]\!] (S)$ corresponds to a particular node a' in the parse tree. If a' is a descendant of a, then each factor corresponding to a' appears in $\text{Expand} [\![\text{Subexpr}_a [\![E]\!]]\!] (N_i)$. Otherwise, it appears in $\text{Expand} [\![E_{\text{sev}}]\!] (S)$. ■

Proposition 3.49. *For any $E, E' \in \mathcal{L}$ where $\text{Sever}_a [\![E]\!] = \text{Sever}_a [\![E']\!] = (N_i, E_{\text{sev}})$, it holds that*

$$\text{Prior} \left[\!\left[E'\right]\!\right] = \text{Prior} [\![E]\!] \cdot \frac{\text{Expand} [\![\text{Subexpr}_a [\![E']\!]]\!] (N_i)}{\text{Expand} [\![\text{Subexpr}_a [\![E]\!]]\!] (N_i)}. \tag{3.50}$$

«

Proof. Use $\text{Prior} [\![E]\!] = \text{Expand} [\![E]\!] (S)$ and $\text{Prior} [\![E']\!] = \text{Expand} [\![E']\!] (S)$. Invoke Proposition 3.48. ■

Lemma 3.50. *Algorithm 3.3 satisfies Condition 3.15 (posterior invariance).* «

Proof. It suffices to establish detailed balance for $\mathcal{T}$ with respect to the posterior:

$$\text{Post} [\![E]\!] (O) \cdot \mathcal{T}(O, E \to E') = \text{Post} \left[\!\left[E'\right]\!\right] (O) \cdot \mathcal{T}(O, E' \to E) \qquad (E, E' \in \mathcal{L}). \tag{3.51}$$

First, if $\text{Post} [\![E]\!] (O) \cdot \mathcal{T}(O, E \to E') = 0$ then $\text{Post} [\![E']\!] (O) \cdot \mathcal{T}(O, E' \to E) = 0$ as follows. Either $\text{Post} [\![E]\!] (O) = 0$ or $\mathcal{T}(O, E \to E') = 0$. There are two cases.

Case 1. If $\text{Post} [\![E]\!] (O) = 0$ then $\text{Likelihood} [\![E]\!] (O) = 0$.
Thus $\alpha(E', E) = 0$.
Then $\mathcal{T}(O, E' \to E) = 0$.

Case 2. If $\mathcal{T}(O, E \to E') = 0$ then $\mathcal{T}(O, E \to E'; a_{\text{root}}) = 0$.
Thus, either $\alpha(E, E') = 0$ or $\text{Expand} \left[\!\left[\text{Subexpr}_{a_{\text{root}}} [\![E']\!]\right]\!\right] (S) = 0$.
If $\alpha(E, E') = 0$ then $\text{Likelihood} [\![E']\!] (O) = 0$ and $\text{Post} [\![E']\!] (O) = 0$.
But $\text{Expand} [\![\text{Subexpr}_{E'} [\![a_{\text{root}}]\!]]\!] (S) = 0$ is a contradiction since

$$\text{Expand} [\![\text{Subexpr}_{E'} [\![a_{\text{root}}]\!]]\!] (S) = \text{Expand} \left[\!\left[E'\right]\!\right] (S) = \text{Prior} \left[\!\left[E'\right]\!\right] > 0.$$

Next, consider $\text{Post} [\![E]\!] (O) \cdot \mathcal{T}(O, E \to E') > 0$ and $\text{Post} [\![E']\!] (O) \cdot \mathcal{T}(O, E' \to E) > 0$. It suffices to show that $\text{Post} [\![E]\!] (O) \cdot |A_{E'}| \cdot \mathcal{T}(O, E \to E'; a) = \text{Post} [\![E']\!] (O) \cdot |A_E| \cdot \mathcal{T}(O, E' \to E; a)$ for all $a \in A_E \cap A_{E'}$. If $E = E'$, then this statement is vacuously true. If $E \neq E'$, there are two cases:

Case 1. If $\text{Sever}_a [\![E]\!] = \text{Sever}_a [\![E']\!] = (N_i, E_{\text{sev}})$ for some i and E_{sev}, then it suffices to show that

$$\text{Post} [\![E]\!] (O) \cdot |A_{E'}| \cdot \text{Expand} \left[\!\left[\text{Subexpr}_a \left[\!\left[E'\right]\!\right]\right]\!\right] (N_i) \cdot \alpha(E, E') \tag{3.52}$$

$$= \text{Post} \left[\!\left[E'\right]\!\right] (O) \cdot |A_E| \cdot \text{Expand} [\![\text{Subexpr}_a [\![E]\!]]\!] (N_i) \cdot \alpha(E', E) \tag{3.53}$$

Both sides are nonzero because

$$\text{Post}\,[\![E]\!]\,(O) > 0 \implies \text{Likelihood}\,[\![E]\!]\,(O) > 0 \implies \alpha(E', E) > 0 \tag{3.54}$$

by Proposition 3.47, and similarly for $\text{Post}\,[\![E']\!]\,(O) > 0$. It suffices to show that

$$\frac{\alpha(E, E')}{\alpha(E', E)} = \frac{\text{Post}\,[\![E']\!]\,(O) \cdot |A_E| \cdot \text{Expand}\,[\![\text{Subexpr}_a\,[\![E]\!]]\!]\,(N_i)}{\text{Post}\,[\![E]\!]\,(O) \cdot |A'_E| \cdot \text{Expand}\,[\![\text{Subexpr}_a\,[\![E']\!]]\!]\,(N_i)} \tag{3.55}$$

$$= \frac{\text{Prior}\,[\![E']\!] \cdot \text{Likelihood}\,[\![E']\!]\,(O) \cdot |A_E| \cdot \text{Expand}\,[\![\text{Subexpr}_a\,[\![E]\!]]\!]\,(N_i)}{\text{Prior}\,[\![E]\!] \cdot \text{Likelihood}\,[\![E]\!]\,(O) \cdot |A'_E| \cdot \text{Expand}\,[\![\text{Subexpr}_a\,[\![E']\!]]\!]\,(N_i)} \tag{3.56}$$

$$= \frac{\text{Likelihood}\,[\![E']\!]\,(O) \cdot |A_E|}{\text{Likelihood}\,[\![E]\!]\,(O) \cdot |A'_E|}, \tag{3.57}$$

where the last uses Proposition 3.49 to expand $\text{Prior}\,[\![E']\!]$, followed by cancellation of factors. Note that if $\alpha(E, E') < 1$ then $\alpha(E', E) = 1$ and

$$\frac{\alpha(E, E')}{\alpha(E', E)} = \frac{\alpha(E, E')}{1} = \frac{\text{Likelihood}\,[\![E']\!]\,(O) \cdot |A_E|}{\text{Likelihood}\,[\![E]\!]\,(O) \cdot |A'_E|}. \tag{3.58}$$

If $\alpha(E', E) < 1$ then $\alpha(E, E') = 1$ and

$$\frac{\alpha(E, E')}{\alpha(E', E)} = \frac{1}{\alpha(E', E)} = \frac{\text{Likelihood}\,[\![E']\!]\,(O) \cdot |A_E|}{\text{Likelihood}\,[\![E]\!]\,(O) \cdot |A'_E|}. \tag{3.59}$$

If $\alpha(E, E') = 1 = \alpha(E', E)$ then $\alpha(E, E')/\alpha(E', E) = 1 = \alpha(E, E')$.

Case 2. If $\text{Sever}_a\,[\![E]\!] \neq \text{Sever}_a\,[\![E']\!]$, then $\mathcal{T}(O, E \to E'; a) = \mathcal{T}(O, E' \to E; a) = 0$.

If $E \neq E'$, then $\mathcal{T}(O, E \to E'; a) = \text{Prior}\,[\![\text{Subexpr}_a\,[\![E']\!]]\!] \cdot \alpha(E, E')$. Finally, posterior invariance follows from detailed balance:

$$\textstyle\sum_{E \in \mathcal{L}} \text{Post}\,[\![E]\!]\,(O)\mathcal{T}(O, E \to E') = \sum_{E \in \mathcal{L}} \text{Post}\,[\![E']\!]\,(O)\mathcal{T}(O, E' \to E) = \text{Post}\,[\![E']\!]\,(O). \tag{3.60}$$

■

Lemma 3.51. *Algorithm 3.3 satisfies Condition 3.16 (irreducibility).* «

Proof. It suffices to show that for all $E \in \mathcal{L}$ and for all $E' \in \mathcal{L}$ such that $\text{Post}\,[\![E']\!]\,(O) > 0$, E' is reachable from E in one step, i.e., $\mathcal{T}(O, E \to E') > 0$. By the definition of a_{root}, it holds that for all $E, E' \in \mathcal{L}$, $a_{\text{root}} \in A_E \cap A_{E'}$ and $\text{Sever}_{a_{\text{root}}}\,[\![E]\!] = \text{Sever}_{a_{\text{root}}}\,[\![E']\!] = (S, \triangle)$. Further, since $\text{Post}\,[\![E']\!]\,(O) > 0$, it must hold that $\text{Prior}\,[\![E']\!] = \text{Expand}\,[\![E']\!]\,(S) > 0$ and $\text{Likelihood}\,[\![E']\!]\,(O) > 0$, which together imply $\alpha(E, E') > 0$. Finally,

$$\mathcal{T}(O, E \to E') \geq \frac{1}{|A_E|} \cdot \mathcal{T}(O, E \to E'; a) \geq \text{Expand}\left[\!\left[E'\right]\!\right](S) \cdot \alpha(E, E') > 0. \tag{3.61}$$

■

Lemma 3.52. *Algorithm 3.3 satisfies Condition 3.17 (aperiodicity).* «

Proof. For all $E \in \mathcal{L}$,

$$\mathcal{T}(O, E \to E) \geq (1/|A_E|) \cdot \text{Expand}\,[\![E]\!]\,(S) = (1/|A_E|) \cdot \text{Prior}\,[\![E]\!] > 0 \tag{3.62}$$

The inequality derives from choosing only $a = a_{\text{root}}$ in the sum over $a \in A_E$ in Eq. (3.47). ■

(r_{11})	$N_1 ::= \texttt{(NoisyGP } N_2 \ N_3\texttt{)}$	
(r_{21})	$N_2 ::= \texttt{(Noise } \square\texttt{)}$	$(\Theta_{21}, \Sigma_{21}, \mu_{21}) = (\mathbb{R}, B(\mathbb{R}), \text{Gamma}(1,1))$
(r_{31})	$N_3 ::= \texttt{(Constant } \square\texttt{)}$	$(\Theta_{31}, \Sigma_{31}, \mu_{31}) = (\mathbb{R}, B(\mathbb{R}), \text{Uniform}(0,10))$
(r_{32})	$\mid \texttt{(Linear } \square\texttt{)}$	$(\Theta_{32}, \Sigma_{32}, \mu_{32}) = (\mathbb{R}, B(\mathbb{R}), \text{Uniform}(-10,10))$
(r_{33})	$\mid \texttt{(Smooth } \square\texttt{)}$	$(\Theta_{33}, \Sigma_{33}, \mu_{33}) = (\mathbb{R}, B(\mathbb{R}), \text{Uniform}(0,10))$
(r_{34})	$\mid \texttt{(Periodic } \square\ \square\texttt{)}$	$(\Theta_{34}, \Sigma_{34}, \mu_{34}) = (\mathbb{R}^2, B(\mathbb{R}^2), \text{Uniform}(0,10) \cdot \text{Uniform}(0,10))$
(r_{35})	$\mid \texttt{(*} \ N_3 \ N_3\texttt{)}$	
(r_{36})	$\mid \texttt{(+} \ N_3 \ N_3\texttt{)}$	
(r_{37})	$\mid \texttt{(Changepoint } \square\ \square \ N_3 \ N_3\texttt{)}$	$(\Theta_{37}, \Sigma_{37}, \mu_{37}) = (\mathbb{R}^2, B(\mathbb{R}^2), \text{Uniform}(-10,10) \cdot \text{Uniform}(0,10))$

Listing 3.2: Context-free grammar expressing the Gaussian process DSL for univariate time series.

Lemmas 3.50–3.52 together establish that the transition operator in Algorithm 3.3 satisfies Conditions 3.15–3.17, which makes it a sound transition operator for synthesis using MCMC (Section 3.3.1) and resample-move SMC (Section 3.3.2) in any probabilistic DSL defined by a context-free grammar.

3.5 Formalizing the Gaussian Process DSL

Chapter 2 demonstrated a probabilistic DSL for Gaussian process models of univariate time series data, whose formal definition using a context-free grammar is shown in Listing 3.2. The Prior semantics are as described in Section 3.4.3, which applies directly to any probabilistic DSL defined by a context-free grammar. Recalling Definition 3.1, the input space $T ::= \uplus_{n=1}^{\infty} \mathbb{R}^n$ consists of finite sequences of real numbers. For any input $t ::= (t_1, \ldots, t_n) \in T$ the output space $\mathcal{X}_t ::= \mathbb{R}^n$ contains the corresponding time series values $x ::= (x_1, \ldots, x_n)$, the sigma-algebra $\mathcal{F}_t ::= B(\mathbb{R}^n)$ and λ_t is the n-dimensional Lebesgue measure. The Likelihood semantics are

$$\text{Likelihood}\,[\![\texttt{(NoisyGP (Noise } \epsilon\texttt{) } K\texttt{)}]\!]\,(t,x) ::= \exp\left(-\frac{1}{2}\Big[x^\top W x - \log\left(\det(W)\right) - n\log\left(2\pi\right)\Big]\right), \tag{3.63}$$

where $K \in \mathcal{L}(G, N_3)$ is a covariance expression and W is the $n \times n$ covariance matrix encoded by (ϵ, K) whose entries $W_{ij} ::= [\![K]\!]\,(t_i, t_j) + \mathbf{1}[t = t'](\epsilon + \epsilon_{\min})$ $(1 \le i, j \le n)$ are obtained from the covariance semantics in Listing 2.2. The parameter $\epsilon_{\min} > 0$ is a "noise floor" that ensure the likelihood is bounded. The next two lemmas establish that the Gaussian process DSL satisfies the conditions needed for Bayesian synthesis described in Problem 3.13 to be well defined.

Proposition 3.53. *The* Prior *and* Likelihood *semantics of the Gaussian process DSL satisfy Conditions 3.6–3.9.* «

Proof for Condition 3.6. Since the non-terminals N_1 and N_2 in Listing 3.2 are non-recursive, it suffices to show that the language generated by the production rule N_3 satisfies the condition for consistency

in Theorem 3.35. A Chomsky normal form of this production rule is

$$
\begin{aligned}
N_3 &\to \underset{7/40}{H_1H_0} \mid \underset{7/40}{H_2H_0} \mid \underset{7/40}{H_3H_0} \mid \underset{7/40}{H_4H_0} \mid \underset{4/40}{H_5N_3} \mid \underset{4/40}{H_6N_3} \mid \underset{4/40}{H_7N_3} \\
H_0 &\to \square \\
H_1 &\to \texttt{Constant} \\
H_2 &\to \texttt{Linear} \\
H_3 &\to \texttt{Smooth} \\
H_4 &\to H_8H_0 \\
H_5 &\to H_9N_3 \\
H_6 &\to H_{10}N_3 \\
H_7 &\to H_{11}N_3 \\
H_8 &\to \texttt{Periodic} \\
H_9 &\to \texttt{*} \\
H_{10} &\to \texttt{+} \\
H_{11} &\to H_{12}H_0 \\
H_{12} &\to H_{13}H_0 \\
H_{13} &\to \texttt{ChangePoint}.
\end{aligned}
$$

Following the notation from Gecse and Kovács [2010, Eq. (2)], define:

$$
\begin{aligned}
m_{ij} = \textstyle\sum_{k=1}^{r_i} p_{ik} n_{ikj}, \text{ where} \qquad r_i &::= \text{ number of productions with } N_i \in N \text{ premise}, \\
p_{ik} &::= \text{ probability assigned to production } k \text{ of } N_i, \\
n_{ikj} &::= \text{ number of occurrence of } N_j \in N \text{ in production } k \text{ of } N_i.
\end{aligned}
$$

The expectation matrix $M ::= [m_{ij}]$ is therefore

$$
\begin{array}{c}
 \\
N \\
H_0 \\
H_1 \\
H_2 \\
H_3 \\
H_4 \\
H_5 \\
H_6 \\
H_7 \\
H_8 \\
H_9 \\
H_{10} \\
H_{11} \\
H_{12} \\
H_{13}
\end{array}
\begin{array}{c}
\begin{array}{ccccccccccccccc}
N_3 & H_0 & H_1 & H_2 & H_3 & H_4 & H_5 & H_6 & H_7 & H_8 & H_9 & H_{10} & H_{11} & H_{12} & H_{13}
\end{array} \\
\left[\begin{array}{ccccccccccccccc}
12/40 & 28/40 & 7/40 & 7/40 & 7/40 & 7/40 & 4/40 & 4/40 & 4/40 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0
\end{array}\right]
\end{array}.
$$

The nonzero eigenvalues of M are $0.71789089\ldots$ and $-0.41789089\ldots$ which lie in the interval $[-1, 1]$. The conclusion follows from Theorem 3.35. ■

Proof for Conditions 3.7 and 3.8. Since $\epsilon_{\min} > 0$, the Likelihood semantic function is the density of a non-degenerate multivariate normal, which is normalized and has full support over its domain. ■

Proof for Condition 3.9. Let $K \in \mathcal{L}(G, N_3)$ be a covariance expression, $C ::= [[\![K]\!](t_i, t_j)]_{i,j=1}^n$ be the $n \times n$ covariance matrix induced by K, and I the $n \times n$ identity matrix. Eq. (3.63) is then

$$
\begin{aligned}
&\text{Likelihood}\,[\![\texttt{(NoisyGP (Noise } \epsilon\texttt{) } K\texttt{)}]\!]\,(t, x) \\
&::= \exp\left(-\frac{1}{2}\Big[x^\top [C + (\epsilon + \epsilon_{\min})I]^{-1} x - \log\left(\det(C + (\epsilon + \epsilon_{\min})I)\right) - n\log(2\pi)\Big]\right) && (3.64)\\
&= ((2\pi)^n \det(C + (\epsilon + \epsilon_{\min})I))^{-1/2} \exp\Big(x^\top [C + (\epsilon + \epsilon_{\min})I]^{-1} x\Big) && (3.65)\\
&\leq ((2\pi)^n \det(C + (\epsilon + \epsilon_{\min})I))^{-1/2} && (3.66)\\
&\leq (\det(C + (\epsilon + \epsilon_{\min})I))^{-1/2} && (3.67)\\
&\leq (\det(C) + \det((\epsilon + \epsilon_{\min})I))^{-1/2} && (3.68)\\
&\leq (\det((\epsilon + \epsilon_{\min})I))^{-1/2} && (3.69)\\
&= \sqrt{\epsilon + \epsilon_{\min}} && (3.70)\\
&\leq \sqrt{\epsilon_{\min}}, && (3.71)
\end{aligned}
$$

where Eq. (3.66) follows from the positive semi-definiteness of $C + (\epsilon + \epsilon_{\min})I$ and the identity $e^{-z} < 1$ for $z > 0$; Eq. (3.67) from $(2\pi)^{-n/2} < 1$; Eq. (3.68) from Minkowski's determinant inequality [Marcus and Minc, 1992, Theorem 4.1.8]; Eq. (3.69) from positive semi-definiteness of C; Eq. (3.71) from $\epsilon > 0$. As (K, ϵ) are arbitrary and the upper bound $\sqrt{\epsilon_{\min}}$ is independent of these terms, the result follows. ■

Since the Gaussian process DSL is generated by a context-free grammar, the transition operator from Algorithm 3.3 is sound by Lemmas 3.50–3.52 and can be used within the MCMC (Algorithm 3.1) and the SMC (Algorithm 3.2) synthesis procedures described in Section 3.3.

3.5.1 Time Complexity Analysis

Each iteration of the transition operator in Algorithm 3.3 takes as an input existing DSL expression $E ::= \texttt{(NoisyGP (Noise } \epsilon\texttt{) } K\texttt{)}$ and proposes a new subexpression $E' ::= \texttt{(NoisyGP (Noise } \epsilon'\texttt{) } K'\texttt{)}$ by applying the following operations:

1. Severing the parse tree at a randomly selected node using Sever (Algorithm 3.3, lines 2 and 3). The cost is linear in the number of subexpressions in E, which is denoted $|E|$.

2. Sampling the PCFG using Expand (3.41) (Algorithm 3.3, line 4). The expected cost is linear in the average length ℓ_G of strings generated by the PCFG, which can be determined from the transition probabilities [Gecse and Kovács, 2010, Eq. (3)]).

3. Assessing the likelihood under the existing and proposed expressions using Likelihood $[\![E]\!]\,(O)$ (Algorithm 3.3, line 6). For a Gaussian process with n observed time points, the cost of constructing the covariance matrix $[[\![K]\!](t_i, t_j)]_{i,j=1}^n$ is $O(|K|n^2)$ and computing the inverse and determinant is $O(n^3)$.

The overall time complexity of an iteration of MCMC or rejuvenation step in SMC is thus $O(\ell_G + \max(|K|, |K'|)n^2 + n^3)$. This term is typically dominated by the n^3 cost of assessing Likelihood. There is a broad literature in sparse approximations to Gaussian processes that reduce this complexity to $O(m^2 n)$ where $m \ll n$ is a parameter representing the size of a subsample of the full data to be used [Rasmussen and Williams, 2006, Chapter 8]. These approximations trade off predictive accuracy with substantial

improvements in scalability. Quiñonero-Candela and Rasmussen [2005] show that many sparse Gaussian process approximations correspond to well-defined probabilistic models that have different likelihood functions than the standard Gaussian process, and so the Bayesian synthesis framework can naturally incorporate sparse Gaussian processes by adapting the Likelihood semantics. As for the quadratic factor $O(\max(|K|, |K'|)n^2)$, this scaling depends largely on the observed data: simpler patterns can be described fewer components in the covariance expression whereas complex patterns require larger covariance expressions.

3.6 Related Work

Synthesizing programs from data is a longstanding activity in computer science. This chapter has established the formal foundations for Bayesian synthesis of programs in both general and context-specific probabilistic DSLs. Chapter 2 demonstrated Bayesian synthesis for univariate time series data in a probabilistic context-free DSL and Chapters 4–6 present Bayesian synthesis techniques in context-sensitive DSLs for cross-sectional data, multivariate time series, and relational data. It should be noted that the terminology of "Bayesian synthesis" in this thesis is to be understood in the sense used in programming languages, and should not be confused with unrelated uses of the term such as in Givens et al. [1994], Green et al. [2000], McAlinn and West [2019], and others.

This section gives a broad literature overview of related work in five related areas: (i) Bayesian synthesis of probabilistic programs where the formalism and proofs in this chapter, a subset of which appear in Saad et al. [2019a], are the first; (ii) non-Bayesian synthesis of probabilistic programs; (iii) probabilistic synthesis of non-probabilistic programs; (iv) non-probabilistic synthesis of non-probabilistic programs; (v) probabilistic structure discovery in machine learning.

Bayesian Synthesis of Probabilistic Programs

Within the literature of Bayesian synthesis, the main contributions of this chapter include

(1) Formalizing probabilistic domain-specific data modeling languages, where each expression specifies the structure and parameters of a given model within a family of probabilistic models for a given domain.

(2) Identifying sufficient conditions on the prior and likelihood semantics of probabilistic expressions needed to obtain a well-defined posterior distribution over DSL programs.

(3) Identifying sufficient conditions to obtain a sound Bayesian synthesis algorithm.

(4) Defining a general family of domain-specific languages generated by context-free grammars with default semantics that ensure the posterior distribution is well defined.

(5) Presenting sound synthesis algorithms using Markov chain Monte Carlo and sequential Monte Carlo that apply to any language generated by a context-free grammar.

(6) Presenting a specific domain-specific language—the Gaussian process language for modeling time series data reviewed in Chapter 2—that is verified to satisfy the required soundness conditions.

Nori et al. [2015] introduce PSKETCH, a system designed to complete partial sketches of probabilistic programs for modeling tabular data. The technique is based on applying sequences of program mutations to a base program written in a probabilistic sketching language. Specifically, Nori et al. [2015] propose to use Metropolis-Hastings sampling obtain maximum a posteriori (MAP) solutions to the sketching problem, where the data likelihood is approximated using a mixture of Gaussians. However, the paper does not establish that the prior or posterior distributions on programs are well defined.

In particular, PSKETCH is formalized to use a uniform prior distribution over an unbounded space of programs, even though there is no valid uniform probability measure over this space. Because the posterior is defined using the prior and the marginal likelihood of all datasets is not shown to be finite, the posterior distribution over sketches is also not well defined. The paper also asserts that the synthesis algorithm converges because the MH algorithm always converges, but it does not establish that the PSKETCH framework satisfies the properties required for MH to converge. And because the posterior is not well defined, there is no probability distribution to which the MH algorithm can converge. Moreover, while the paper claims to use MH to determine whether a proposed program mutation is accepted or rejected, there is no tractable algorithm that can be used to compute the reverse probability of going back from a proposed program to an existing program, which means that MH cannot be used with the program mutation proposals described in the paper.

Moreover, the PSKETCH system in Nori et al. [2015] is based on a probabilistic sketching language with constructs drawn from general-purpose programming languages, including such as if-then-else, for loops, variable assignment, and arithmetic operations. Working with these general constructs produces a search space that is too unconstrained to yield practical solutions to real-world data modeling problems, especially in the absence of other sources of information that can more effectively narrow the search. As with traditional synthesis, domain-specific languages such as the ones presented in this thesis make it possible to effectively solve automatic data modeling problems. For example, while the PSKETCH language in Nori et al. [2015] contains general-purpose programming constructs, it does not have domain-specific constructs that compactly encapsulate Gaussian processes or valid covariance functions.

Hwang et al. [2011] use beam search over programs in a subset of the Church probabilistic programming language [Goodman et al., 2008a] for synthesizing tree-like s-expressions. The resulting search space is so unconstrained that, as the authors note in the conclusion, the synthesis technique does not easily scale real-world problems. This drawback highlights the benefit of controlling the search space via an appropriate domain-specific data modeling language. Although Hwang et al. [2011] also use the vocabulary of Bayesian synthesis, the proposed program-length prior over an unbounded program space is not shown to always be probabilistically well formed and may not lead to a valid Bayesian posterior distribution over programs. In addition, the stochastic approximation of the program likelihood does not result in a sound synthesis algorithm.

Several papers in the cognitive science literature have used Bayesian inference to synthesize expressions in probabilistic domain-specific languages of intuitive theories [Ullman and Tenenbaum, 2020]. For example, Goodman et al. [2008b] synthesize logical rules in a "concept language" generated by a context-free grammar; Ullman et al. [2012] synthesize Horn clauses generated by a probabilistic Horn clause grammar; and Ullman et al. [2018] synthesize Church expressions that define rudimentary rules of intuitive physics. Cranefield and Dhiman [2021] apply the techniques in this chapter to synthesize programs for identifying social norms that govern agent interactions. Inference in these works is performed using Markov Chain Monte Carlo sampling. Many of these works can be seen as specific instantiations of the general framework presented in Section 3.4 for Bayesian synthesis over probabilistic DSLs generated by context-free languages.

Non-Bayesian Synthesis of Probabilistic Programs

Ellis et al. [2015] introduce a method for synthesizing probabilistic programs by using SMT solvers to optimize the likelihood of the observed data with a regularization term that penalizes the program length. The research works with a domain-specific language for morphological rule learning. Perov and Wood [2014] propose to synthesize code for simple random number generators using approximate Bayesian computation. These two techniques are fundamentally different from the Bayesian synthesis problem presented in this chapter. Neither technique is based on Bayesian inference and neither presents soundness proofs or states a soundness property. Moreover, both approaches attempt to find a single

highest-scoring program as opposed to synthesizing ensembles of programs that approximate a posterior distribution over probabilistic programs. As this previous research demonstrates, obtaining soundness proofs or characterizing the uncertainty of the synthesized programs against the data generating process is particularly challenging for non-Bayesian approaches.

Lake et al. [2015] learn probabilistic programs that solve challenging one-shot learning of visual concepts for recognizing handwritten characters. Due to the sophisticated nature of the prior distribution over probabilistic programs that produce handwritten characters, inference is performed using a combination of discrete approximations, continuous optimization, and MCMC sampling [Lake et al., 2015, supplementary material, Section S3], which together trade off soundness for runtime performance. Guimerá et al. [2020] use a Bayesian framework for synthesizing mathematical expressions of noisy regression functions and apply the technique to discover models from finance, systems biology, and physics data. The prior over mathematical expressions is estimated from a corpus of 4080 expressions mined from Wikipedia. The likelihood that an expression assigns to observed data is approximated using the Bayesian information criterion (BIC) heuristic. It is fruitful to explore resample-move sequential Monte Carlo learning algorithms from Section 3.3.2 for more principled, scalable, and fully-Bayesian probabilistic program synthesis in these important problems.

Tong and Choi [2016] describe a method which uses the off-the-shelf model discovery system from Lloyd [2014] to learn Gaussian process models and the render the models in the Stan [Carpenter et al., 2017] probabilistic programming language. However, the approach in Tong and Choi [2016] does not formalize the program synthesis problem; nor does it present any formal semantics; nor is it able to extract posterior probabilities that qualitative properties hold in the data; nor does it apply to multiple model families in a single problem domain, let alone multiple problem domains. Chasins and Phothilimthana [2017] present a technique for synthesizing probabilistic programs of tabular datasets in a subset of BLOG [Milch et al., 2005]. Unlike the probabilistic DSL for cross-sectional tabular data from Chapter 4, this approach requires the user to manually specify a causal ordering among the variables. This knowledge is rarely available for real-world data, and it is not obvious how to infer causal orderings from observational data. Second, the technique of Chasins and Phothilimthana [2017] is based on using linear correlation, which fail adequately capture complex relationships. Finally, the synthesis technique uses simulated annealing not Bayesian learning, has no probabilistic interpretation, approximates the program likelihood using mixtures of Gaussians, and does not provide a notion of soundness.

Probabilistic Synthesis of Non-probabilistic Programs

Schkufza et al. [2013] describe a technique for synthesizing high-performance X86 binaries by using MCMC to stochastically search through a space of programs. The output is a single X86 program which satisfies the hard constraint of correctness and the soft constraint of performance improvement. While both the Bayesian synthesis framework in this chapter and the superoptimization technique from Schkufza et al. [2013] can leverage MCMC algorithms to implement the synthesis, they use MCMC in a fundamentally different way. In Schkufza et al. [2013], MCMC is used to search through a space of deterministic programs that seek to minimize a cost function that has no probabilistic semantics. In contrast, Bayesian synthesis uses MCMC (or SMC) to approximately sample from a space of probabilistic programs whose semantics specify a well-defined posterior distribution over programs.

Bayesian approaches for sampling from a posterior distribution over deterministic programs have also been investigated. Liang et al. [2010] use adapter grammars [Johnson et al., 2006] to build a hierarchical nonparametric Bayesian prior over programs specified in combinatory logic [Schönfinkel, 1924]. Ellis et al. [2016] describe a method to sample from a bounded program space with a uniform prior where the posterior probability of a program is equal to zero if it does not satisfy an observed input-output constraint, or geometrically decreasing in its length otherwise. This method is used to synthesize arithmetic operations, text editing routines, and list manipulation programs. Both Liang

et al. [2010] and Ellis et al. [2016] specify prior distributions over programs similar to the prior in this chapter. However, these two techniques assume that the synthesized programs have deterministic input-output behavior and cannot be easily extended to synthesize programs that have probabilistic input-output behavior.

Lee et al. [2018] present a technique to speed up program synthesis of non-probabilistic programs, where A* search is used to enumerate programs that satisfy a set of input-output constraints in order of decreasing prior probability. The prior distribution over programs is itself learned using a probabilistic higher-order grammar. The technique is used to synthesize programs in domain-specific languages for bit-vector, circuit, and string manipulation tasks. Similar to the Bayesian synthesis framework, Lee et al. [2018] use PCFG priors to specify domain-specific languages. However the fundamental differences are that the synthesized programs in Lee et al. [2018] are non-probabilistic and the objective is to enumerate valid programs sorted by their prior probability, while in Bayesian synthesis the programs are probabilistic and the objective is to sample programs according to their posterior probabilities.

Non-probabilistic synthesis of non-probabilistic programs

Over the last decade program synthesis has become a highly active area of research in programming languages, and a full literature review is too vast to summarize here. Key techniques include deductive logic with program transformations [Burstall and Darlington, 1977, Manna and Waldinger, 1979, 1980], genetic programming [Koza, 1992, Koza et al., 1997], solver and constraint-based methods [Solar-Lezama et al., 2006, Gulwani et al., 2011, Gulwani, 2011, Alur et al., 2013, Feser et al., 2015, Kim et al., 2021], and neural networks [Graves et al., 2014, Reed and de Freitas, 2016, Balog et al., 2017, Chaudhuri et al., 2021]. These approaches have generally focused on areas where uncertainty is not essential or relevant to the problem being solved. Synthesis tasks in these works apply to programs that exhibit deterministic input-output behavior, typically for discrete problem domains. The usual formulation is to define an "optimal" solution and the goal of program synthesis to find this solution or some approximation of it. In contrast, in Bayesian probabilistic program synthesis, the problem domain is fundamentally about automatically learning models of non-deterministic data generating processes given a set of noisy observations. Given this characteristic of the problem domain, the solutions from Bayesian synthesis capture uncertainty at two levels. First, each synthesized probabilistic program exhibits noisy, non-deterministic input-output behavior. Second, the ensemble of synthesized programs characterizes uncertainty over the structure and parameters of expressions in the DSL, which is inherent in real-world empirical phenomenon.

Probabilistic Structure Discovery

Researchers have developed several probabilistic techniques for discovering statistical model structures from observed data [Tenenbaum et al., 2011]. Examples include Bayesian network structures [Friedman and Koller, 2003, Mansinghka et al., 2006]; sparse deep graphical models [Adams et al., 2010]; matrix-composition models [Grosse et al., 2012]; multivariate data tables [Mansinghka et al., 2016], further discussed in Chapter 4; univariate time series [Wilson and Adams, 2013, Duvenaud et al., 2013]; multivariate time series [Saad and Mansinghka, 2018], further discussed in Chapter 5; relational data [Kemp et al., 2006, Saad and Mansinghka, 2021], further discussed in Chapter 6. Several of these methods also approach the structure learning problem in terms of a hierarchical Bayesian inference problem. The formalism presented in this chapter helps explain, formalize, and unify these methods in terms of synthesizing programs in probabilistic domain-specific data modeling languages. The formalisms establish general conditions for Bayesian synthesis over structured expressions to be well defined and introduces sound synthesis algorithms that apply to a broad class of DSLs produced by context-free grammars. The learned model ensemble can then be used to compute the approximate posterior probability that various structures are present or absent in the data and a full distribution over predicted values. This

capability rests on a distinctive aspect of the presented formalism, namely that soundness is defined in terms of sampling programs from a distribution that converges to the Bayesian posterior, as opposed to finding a single "highest scoring" program. These capabilities improve upon the Gaussian process learning technique in Duvenaud et al. [2013], for example, which leverages greedy search for inferring covariance structure, does not support online learning with streaming data, and does not rest on any notion of soundness. The workflow in Figure 3.1 shows one way that probabilistic programming languages makes structure discovery techniques more usable. Specifically, the proposed approach is to translate programs from domain-specific languages to probabilistic programming languages (such as SPPL, described in Chapter 7) that have built-in constructs for solving probabilistic inference queries, instead of developing new inference engines for each model class.